zbMATH Web Scraping API

Jefferson Orion Portelli

University of St Andrews

Supervised by
Olexandr Konovalov

28th March, 2022

Abstract

zbMATH and swMATH offer a rich repository of information on mathematics papers and software. Despite this, large-scale exploration of this data is almost entirely unavailable due to the limited functionality offered by their APIs. This project aims to break this barrier by providing users access to this data in an intuitive, easily usable format. To this end, a Python API was developed for web-scraping large quantities of information about different records. Using this system, data analysts with varying levels of technical experience can access mathematical software data in a simple and consistent way. Following this, a use case example of the system for reproducible data analysis is provided to facilitate design and personalisation for future experiments.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated. The main text of this project report is 11,919 words long, including project specification and plan. In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

Contents

1 Introduction **5**
1.1 Impact of UCU Strike Action 5

2 Context Survey **6**
2.1 zbMATH Systems 6
2.1.1 zbMATH Open 6
2.1.2 zbMATH Open API 8
2.1.3 swMATH 9
2.2 Tools & Concepts 10
2.2.1 Web Scraping 10
2.2.2 APIs 11
2.3 Existing Systems 11

3 Requirements Specification **13**
3.1 Initial Requirements 13
3.1.1 Primary Requirements 13
3.1.2 Secondary Requirements 13
3.2 Adapted Requirements 13
3.2.1 Primary Requirements 14
3.2.2 Secondary Requirements 14

4 Software Engineering Process **15**
4.1 Development Strategy 15
4.2 Development Structure 15
4.2.1 Project Scoping 15
4.2.2 API Development 15
4.2.3 Data Collection 16
4.2.4 Data Analysis 16
4.3 Testing 16

5 Ethics **18**

6 Design **19**
6.1 Project Architecture 19
6.2 Scraper 19
6.2.1 Structure 19
6.2.2 Obtaining Record Information 20
6.2.3 Choice of Fields 20
6.2.4 ID Choice 20
6.2.5 MSC Code Detail Level 21
6.3 API 21
6.3.1 Aims 21
6.3.2 Structure 22
6.3.3 Web API & Python API 22

6.3.4 RESTful vs RESTless . . . 22
6.3.5 Allowing Choice of Fields . . . 23
6.3.6 Two Pronged Data Collection . . . 23
6.3.7 Quality-of-Life Functions . . . 23
6.4 Data Collection & Analysis . . . 24

7 Implementation **25**
7.1 Scraper . . . 25
7.1.1 Field Types & Examples . . . 25
7.1.2 Scraping ID . . . 26
7.1.3 Accessing Date . . . 26
7.1.4 Exception Handling . . . 26
7.2 API . . . 27
7.2.1 Flask-RESTful API . . . 27
7.2.2 API Client Considerations . . . 28
7.3 Rate Limits . . . 30
7.4 Testing . . . 32

8 Experiment **33**
8.1 Collecting Data . . . 33
8.1.1 Experiment Scoping . . . 33
8.1.2 Parameter Selection . . . 34
8.1.3 Data Collection . . . 35
8.2 Analysis . . . 36
8.2.1 Visualisation Choice . . . 36
8.2.2 Data Transformation . . . 36
8.2.3 Visualisation . . . 37
8.2.4 Assessment . . . 37

9 Evaluation & Critical Appraisal **38**
9.1 Primary Requirements . . . 38
9.1.1 Requirement 1 . . . 38
9.1.2 Requirement 2 . . . 38
9.1.3 Requirement 3 . . . 39
9.2 Secondary Requirements . . . 39
9.2.1 Requirement 4 . . . 39
9.2.2 Requirement 5 . . . 40
9.2.3 Requirement 6 . . . 40

10 Appendix A - Ethics **43**

11 Appendix B - Instructions **44**

1 Introduction

The role of software in modern mathematics is arguably central, however the exact form this takes can vary wildly between two subjects. While statisticians may use packages in R to fit models and run experiments, graph theorists may use entirely different systems to prove fundamental properties about the chromatic numbers of planar graphs. The integration of software itself may also vary dramatically over time subject by subject, with more applied fields finding natural computational uses early on as opposed to pure subjects. Questions such as these may be deeply important to developers of these computational tools in finding their audience usage statistics in academics.

Fortunately, in the era of information this data is all readily stored. zbMATH Open is currently the largest reviewing and abstracting service for mathematics papers. With over four million articles spanning all the way back from the 18th century to the present, it is certainly the most comprehensive database of mathematics research. More importantly though, zbMATH, in conjunction with its sister site swMATH, has the unique trait of including software citation information of many of these papers in a clearly linked way.

Unfortunately, this information is still largely inaccessible in large quantities, and while swMATH offers some small visualisations of citation history, it is hardly sufficient to answer even the questions posed above. To allow greater accessibility to this novel datasource, this project involves the development of a simple Python API which uses web-scraping to find records specific software information at large scales. This API aims to allow easy access to inexperienced users while also facilitating reproducible and consistent experiments. To demonstrate this we also provide an example experiment showing the collection, cleaning, and visualisation of data at a reasonable scale.

This paper follows the execution of these goals in several steps. First, we look into the required concepts, platforms, and other relevant context of the project. Following this, we formalise and prioritise project goals and objectives. Then we analyse the specific approach, implementation steps, and ultimate completion of the stated objectives, before finally returning to analyse the results in relation to our initial requirements and goals.

1.1 Impact of UCU Strike Action

Due to UCU strike action that occurred during the project, a period of no contact with supervisor Olexandr Konovalov occurred from 7th feb to 14th march. This interval coincided with a critical moment in the project wherein the data collection was prepared to be performed. Due to this, I could not access personalised advice setting up the programs to remotely execute for long durations. Further, this was the same time period the issue of rate limits was uncovered, leading to a delay in the development of workaround. These two issues ultimately led to smaller datasets than expected due to time constraints during scraping.

2 Context Survey

2.1 zbMATH Systems

Naturally, the zbMATH databse and platform is the core of this project, hence we spend time exploring it in detail to understand the extent of its capacity. All information referenced here is based on information available on zbMATH.org directly [1].

2.1.1 zbMATH Open

History

Originally founded in 1931 as a reviewing service for mathematical literature, Zentralblatt für Mathematik und ihre Grenzgebiete—more simply referred to as Zentralblatt—aimed to provide quick reviews for an international selection of mathematical material. As the number of mathematical publications grew in the 1970s, Zentralblatt began to modernise their database using magnetic tapes to keep up with their aim for promptness. With the advent of the internet, in 1996 Zentralblatt was transformed into an internet accessible service known as zbMATH. Finally, zbMATH transitioned to an openly accessible platform in January 2021.

Services

The platform itself is relatively minimalistic, with the initial interface providing only a search line for documents or articles.

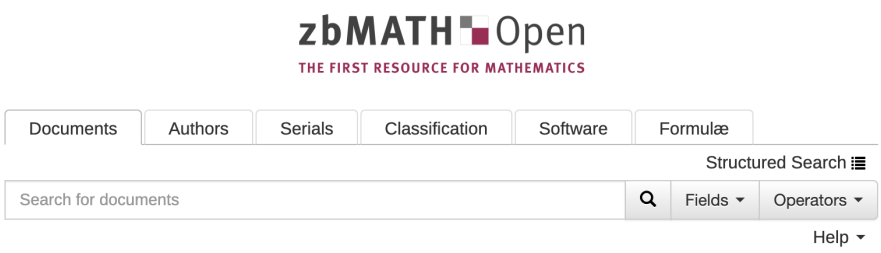
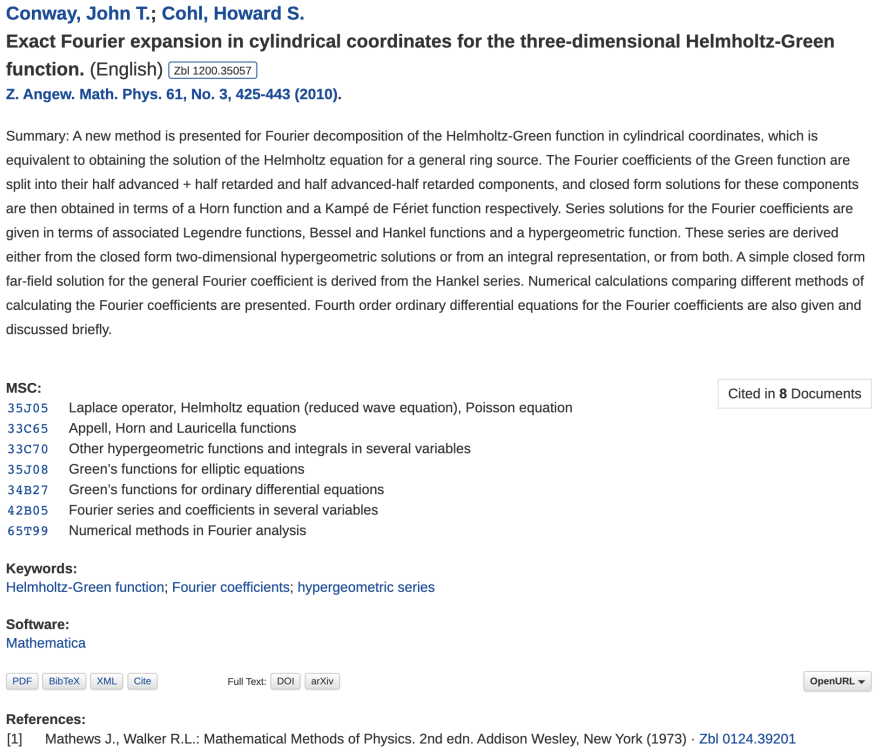


Figure 1: zbMATH.org search feature

As the UI suggests, the site is extremely simple, focused solely on exploring the articles contained therein. Each tab at the top of the search bar simply alters the search function to look for the category of content. This selection of options displays zbMATH the comprehensive collection of information zbMATH stores about the articles it indexes and reviews. Detailed searches can be performed allowing filtering by even more categories.

Record Structure

The primary resource provided by the platform are records. These records represent mathematical publications and each contain an array of summary information.



Conway, John T.; Cohl, Howard S.

Exact Fourier expansion in cylindrical coordinates for the three-dimensional Helmholtz-Green function. (English) Zbl 1200.35057

Z. Angew. Math. Phys. 61, No. 3, 425-443 (2010).

Summary: A new method is presented for Fourier decomposition of the Helmholtz-Green function in cylindrical coordinates, which is equivalent to obtaining the solution of the Helmholtz equation for a general ring source. The Fourier coefficients of the Green function are split into their half advanced + half retarded and half advanced-half retarded components, and closed form solutions for these components are then obtained in terms of a Horn function and a Kampé de Fériet function respectively. Series solutions for the Fourier coefficients are given in terms of associated Legendre functions, Bessel and Hankel functions and a hypergeometric function. These series are derived either from the closed form two-dimensional hypergeometric solutions or from an integral representation, or from both. A simple closed form far-field solution for the general Fourier coefficient is derived from the Hankel series. Numerical calculations comparing different methods of calculating the Fourier coefficients are presented. Fourth order ordinary differential equations for the Fourier coefficients are also given and discussed briefly.

MSC:

Cited in **8** Documents

35J05 Laplace operator, Helmholtz equation (reduced wave equation), Poisson equation
33C65 Appell, Horn and Lauricella functions
33C70 Other hypergeometric functions and integrals in several variables
35J08 Green's functions for elliptic equations
34B27 Green's functions for ordinary differential equations
42B05 Fourier series and coefficients in several variables
65T99 Numerical methods in Fourier analysis

Keywords:
Helmholtz-Green function; Fourier coefficients; hypergeometric series

Software:
Mathematica

PDF BibTeX XML Cite Full Text: DOI arXiv OpenURL

References:
[1] Mathews J., Walker R.L.: Mathematical Methods of Physics. 2nd edn. Addison Wesley, New York (1973) · Zbl 0124.39201

Figure 2: The typical zbMATH record layout

Figure 2 demonstrates the appearance of a typical zbMATH record. This record contains many details about the document it represents, most notably:

Title, Language, Publication Source, Publication Date, Summary, MSC Codes, Keywords, Software, References.

While some variations of these fields can occur, records are generally formatted as the figure display, only with certain fields (such as software) occasionally omitted.

MSC

Mathematics Subject Classification (MSC) is a scheme for categorising mathematical publications by subject. It was last updated in 2020 and is hierarchical

in its level of detail. For example, the code 05 represents *combinatorics* and contains a subset 05CXX, representing graph theory.

2.1.2 zbMATH Open API

Along with the transition to being a freely accessible database, zbMATH also created a new API to aid mathematical researchers. This API provides four main endpoints for accessing information from the database, which are listed below:

GetRecord Retrieves an individual zbMATH entry, as seen above. The structure of this output will be explored in detail shortly.

ListIdentifiers Lists the identifiers for all 4,206,870 records currently in the zbMATH database. Several filter parameters can be used to narrow this set per user requirements.

1. **Set:** The MSC code of the ID's to collect.
2. **Start:** Earliest uploaded record to collect IDs from.
3. **End:** Last uploaded record to collect IDs from.

ListRecords Retrieves all 4,206,870 records in the zbMATH database. Several filter parameters can be used to narrow this set per user requirements (same as above).

ListSets Lists all 63 high level MSC sets zbMATH can filter by.

Record Structure

Each record returned by the zbMATH API contains a more segmented and easily machine readable version of the respective website view. These responses are made using XML and contain the following fields:

Identifier, Authors, Title, Language, Publication Source, Publication Date, Upload Date, MSC Codes (high level & detailed), Keywords, References.

Note the structure is almost identical to the fields provided through the web view, with the most notable exception being the absence of the software field. For the purposes of this project this absence is crucial.

Multi-page Outputs

All requests except GetRecord are capable of returning multiple pages of responses (to avoid returning over 4 million items at once). To facilitate this, the zbMATH API utilises *resumption tokens*, unique strings generated after responses which users must append to subsequent queries to retrieve the next page. These tokens expire within a second and hence require automation to use.

Usage Rights

The zbMATH Open API, as well as the zbMATH dataset as a whole is licensed under CC BY-SA 4.0. This allows users to share and adapt any materials therein under the condition that they are appropriately credited and shared under the same licence [2].

2.1.3 swMATH

swMATH is a freely accessible partner site to zbMATH Open that provides information on a variety of mathematical software packages. It includes both a database of these packages as well as a web of links to zbMATH articles which either discuss or utilise them. The site aims to help software authors understand where their software is utilised as well as help users identify the quality of potential packages.

Site Structure

As with zbMATH, the layout of the site is extremely simple with a simple search bar dominating the landing page. Through this, any mathematical software package can be viewed with the following main appearance:

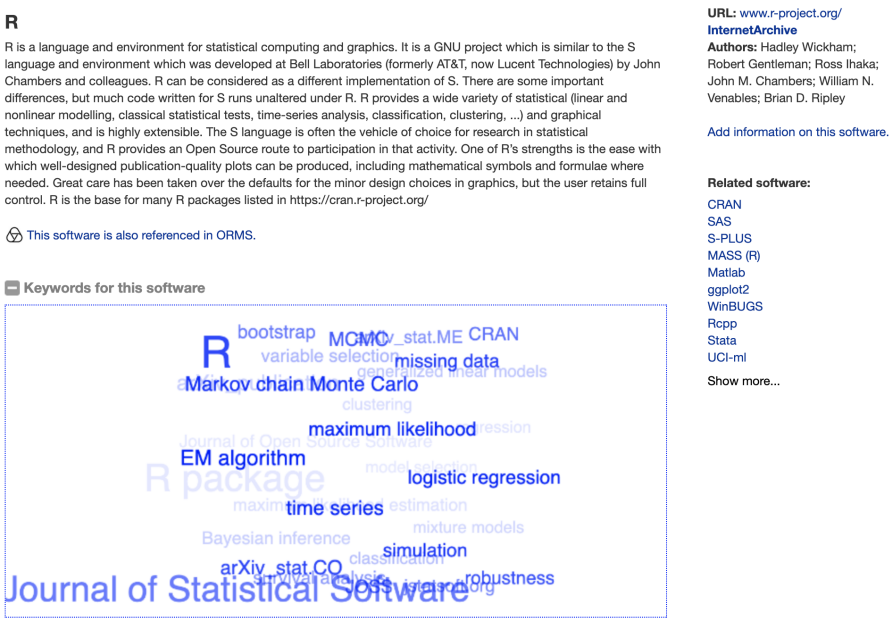


Figure 3: The upper half of the swMATH page for R

As figure 3 shows, the swMATH layout is much more visually oriented than zbMATH, containing the many visualisations. Paired with the lower half of the

page (not show), each entry provides the following key items:

Package Name, Description, Link to the Package, Authors, Related Software, Keyword Visualisation, List of Papers Citing this Package, Most Popular MSC Codes, Visualisation of Usage Frequency Over Time.

2.2 Tools & Concepts

2.2.1 Web Scraping

Typically, obtaining information through websites can be done through either dedicated API's or manual browsing. Using zbMATH as an example, the easiest way to access information about the database is through their dedicated API. Unfortunately, not all websites provide an API, and not all APIs provide all the necessary information. For example, the zbMATH API provides an array of information about records in a clean and easily machine readable XML format. Despite this, it does not contain one crucial piece of information: software.

Fortunately, software itself is clearly available, as browsing through the web interface clearly shows us a tab for software (figure 2). Clearly the information is there, but how do we access it? The answer to this is web scraping.

Definition & Structure

Defined as "the process of extracting and combining contents of interest from the Web in a systematic way" [5], web scraping offers an invaluable approach to obtaining web-based content often obscured from automation. Put more simply the process of web scraping operates in three main steps:

1. The page of interest is requested using the standard HTTP GET request. This action is identical to loading a page in the browser and retrieving its HTML. Note, some sites have systems in place to prevent automated bot access.

2. The HTML response is parsed and the fields of interest are extracted. This phase may include simple searching through tags and regex or more complicated text mining techniques.

3. The extracted contents are formatted for output.

Due to the precise nature of this text mining, many web scrapers are extremely weak to changes in site structure [5].

Access Prevention

Naturally, many sites are loath to allow bots access and implement various techniques to identify and prevent these intrusions. Most commonly, these intrusion prevention techniques take the form of rate limits, hard thresholds on the number of requests able to be made in a given interval. These can pose large obstacles

to web scrapers, as staying beneath the limits can slow the process down dramatically. zbMATH employs its own limit as a threshold on daily requests. The ramifications of this will be discussed at length during implementation section.

Legal Concerns

Web scraping is also subject to legal scrutiny due to its ambiguous interaction with copyright laws and potentially damaging effect on websites (cite Zhao). Frequent access attempts may also be virtually indistinguishable from D.O.S attacks. As such, maintaining reasonable access frequencies is important to maintaining an ethical scraper. Fortunately, many sites (such as zbMATH) employ their own rate limits to avoid these concerns. Additionally, zbMATH offers explicit licensing to information in their database, so copyright issues are entirely alleviated.

2.2.2 APIs

Application programming interfaces (APIs) provide high level access to potentially complicated applications. These tools expose the end user to reusable and highly abstracted methods to interact with their underlying programs in a unified manner. Most commonly, these take the form of REST web APIs for remote resource access.

REST APIs

Representational State Transfer (REST) is a framework for developing Web services that has defined the development of the Web. RESTful API's access resources via URL and generally output JSON or XML. This framework is generally defined by following principles [7]:

Resource Addressability APIs expose resources from the backend using uniquely addressable *Uniform Resource Identifiers* (URI's).

Representation Separability Users do not need to know or be exposed to the underlying storage representation, and instead work with the representations needed for their purposes (JSON and XML).

Standardised Accessibility Resources are accessed using HTTP protocol methods (Get, Post, Delete, etc.)

Statelessness Interactions between the user and the API store no state. Performing the same request twice will not have different results.

2.3 Existing Systems

Two other Python based API's for interacting with zbMATH have previously been created, neither of which were referenced in this project due to either their obsoletion or irrelevance to the task at hand.

ice-MC2 zbMATH API

This project served as a simple wrapper for the zbMATH search function. It utilised BeautifulSoup and Python to perform a search for matching documents using a provided search term. The provided search term was entered directly into the search line on zbMATH and a list of authors, link, title pairs was returned. Last updated in 2019, this system was made obsolete due to the creation of the zbMATH Open API. [3]

zbMATH Open Links API

Developed by members of zbMATH's parent institution—*FIZ Karlsruhe – Leibniz Institute for Information Infrastructure*—the zbMATH links API aimed to show connections between zbMATH and external platforms which contain links to objects indexed by zbMATH. Last updated 2022/01/05, the project is still being maintained at the time of this report. [4]

3 Requirements Specification

Several objectives were established during the initial DOER for this project. However, to more accurately encompass the shifting needs that arose during development, these evolved naturally throughout the course of the project. The initial objectives follow:

3.1 Initial Requirements

3.1.1 Primary Requirements

1. Develop an API for easily collecting and aggregating data from zbMath & swMath in Python.

2. Identify the relationship between individual fields of mathematics and the software used therein. For example: "what is the most commonly used software in this field?"

3. Design a clear procedure for collecting, cleaning, and analysing the data according to the following principles:

   (a) *Correctness*

   (b) *Repeatability:* Procedure can be repeated from the start to the end with the same machine.

   (c) *Replicability:* Other people can replicate the results on their own machine.

   (d) *Reproducibility:* It should be to work on another dataset with similar structure to analyse the same metrics.

   (e) *Reusability:* It should be possible to reuse some components in different experiments

3.1.2 Secondary Requirements

1. Employ Python & Jupyter Notebooks to create a strong visual presentation of analytical findings in primary objective 2.

2. Investigate which facts about the relationship between different mathematical software packages can be inferred from their citations

3.2 Adapted Requirements

While the overarching goals of developing an API and analysing data about software in mathematics remained the same, priorities were greatly shifted. Initially the focus was data visualisation and analysis, but as the project developed it became clear that the tools were significantly more complex and far more valuable than the analysis. The results of this reprioritization, and hence the final requirements, are restated to reflect this.

3.2.1 Primary Requirements

1. Develop a web scraper capable of mass collection of software information from zbMATH records.

2. Develop an intuitive Python API for integrating the scraping tools into data analytics projects. This API should facilitate procedures which adhere to the following principles:

   (a) *Correctness:* The information obtained through the API should be correct.

   (b) *Repeatability:* The tools yield consistent datasets on the same machine.

   (c) *Replicability:* The tools yield consistent datasets on different machines.

   (d) *Reusability:* The tools are flexible in yielding requirement specific datasets.

3. Demonstrate the use of the aforementioned tools through a small scale procedure (adhering to the five principles set out in the initial requirements).

3.2.2 Secondary Requirements

1. Adapt the API to allow automated, unchecked collection of data beyond rate limits.

2. The example procedure should yield a comprehensive dataset capable of reuse in other data analysis experiments.

3. The example procedure should create a strong visualisation of its findings through Jupyter Notebooks.

The primary requirements represent the minimum conditions that must be met for the project to be considered successful, while the secondary objectives act as additional goals to be achieved if possible.

4 Software Engineering Process

4.1 Development Strategy

This project was developed roughly according to Agile procedures, with iterative development being driven by the aim of constantly having a working system. While there was only a single developer, weekly supervisor meetings served as an opportunity to establish goals, discuss obstacles, and reflect on progress.

Compared to the traditional waterfall model, this iterative approach was much more favourable for this project. This could immediately be seen by the change in requirements addressed in section 3, with a more adaptive development strategy becoming essential. Additionally, due to the reliance on the underlying structure of zbMATH, the scale of the project (and hence the priorities) became more apparent as the site's limitations were explored further.

4.2 Development Structure

The project itself was split into several phases, each including sub-phases which can be thought of analogously to sprints in the traditional Agile development process. At a high level, this consisted of one planning phase and three development phases. See the design and implementation sections for the specific outcomes of each stage. We now define of these phases.

4.2.1 Project Scoping

In this phase, the structure of the project (as described in this section) was established and rough timeframes for each phase were estimated. Additionally, implementation specific tools & frameworks were determined. Due to the adaptive nature of the development process, many decisions made during this phase were liable to significant changes.

4.2.2 API Development

This section formed the bulk of the project as described by primary requirements 1 and 2. It was composed of two components:

Web Scraper

Here, aspects pertaining to structure of zbMATH records, required fields, and those available through the existing API were explored. Subsequently, the web scraper was implemented, which, at a minimum level, collected information about required fields from a record.

API

This section involved the development of the actual interface for collecting data. Initially this took the form of a REST-ful Flask API similar to the one hosted

by zbMATH, however it ultimately evolved into two forms: a REST-ful Python API client, which returned results about records in a stateless manner; and a file-interacting Python client capable of providing simple access to record data collection and storage. This was the primary product of the project.

4.2.3 Data Collection

This phase was marked by the shift towards demonstrating the produced tools and measuring their performance when operating at scale. Moreover, it served as the first step to achieving primary requirement 3. It was broken into three subphases:

Experiment Design

Due to the focus on data collection, it was first necessary to determine the scope of data to be collected for analysis. This included considerations such as scraper speed and rate limits, while also ensuring a sufficient quantity of data was collected to answer any questions that might arise.

Trials & Optimisation

API performance at large scales was unpredictable, so setting up scripts to automate collection and determine issues in the data was essential. This included fixing previous tools to account for variations in the data as well as adding exception handling to ensure the program could be run unmonitored for extended periods.

Data Collection

Finally, the scraper was run and the data was collected over the span of several days.

4.2.4 Data Analysis

The final stage of the project: a relatively minor visual analysis of findings to demonstrate usage of the dataset using Jupyter Notebooks.

4.3 Testing

Since these tools work primarily by making requests to zbMATH, unit testing is not entirely feasible. Despite this, a small number of unit tests were written to test certain smaller features of the system. These could be potential expanded in the future if necessary. Additionally, since the system works with live data, the best way to test its features was through large scale deployment. This was achieved during the data collection phase as a selection of over 50,000 records was scraped for the final result. Ultimately, since this is a data collecting tool (and a scraper particularly), there is necessarily going to be some noise in the

data retrieved. Fortunately, if the software is scraped correctly it is possible to get accurate results for all remaining columns using the provided cleaning functions.

5 Ethics

As per the DOER, this project raises no special ethical concerns. The tools created grant access to data that is already publicly available via zbMATH. Further, the analysis and sample dataset raise no concerns as they focus solely on generalised statistics about mathematical subjects and software; the names of individual authors will not be stored or utilised in any way.

6 Design

As outlined in the section 4, this project was broken into 3 phases. The design will hence be outlined roughly according to this.

6.1 Project Architecture

All design considerations surrounding project architecture were made with respect to an abstract view of a hypothetical data analysts requirements (effectively "user stories"). Throughout this section we refer to this hypothetical individual as the **end-user** or sometimes just the **user**. This serves to align the designs of the systems to the practical needs of its users. Their **goals** are to perform analyses on the use of software in various mathematical fields, and it is the aim of the subsequent sections to facilitate those goals.

While no explicit user stories were created, the following represents a general idea of the forms these considerations could take:

1. User wishes to find all software used for a record in Python
2. User wishes to get record info using an API get request
3. User wishes to get a list of all records within a date range in a set
4. User wishes to get info from all records within a date range in a set
5. User wishes to resume collection of record info if it terminated early
6. User wishes to run data collection for an extended period (24hr+)
7. User wishes to examine relationship between software and time

6.2 Scraper

The aim of primary requirement 1, the *Scraper*, was aimed towards accessing software information about any specific record. Since software record information is not available through the zbMATH API itself, a web scraper is required.

6.2.1 Structure

At an overview, a *Scraper* represents an object which acts on a single zbMATH record's link. This *Scraper* then provides access to various attributes of the record by web-scraping the underlying HTML of the associated page. Systems utilising this tool (such as the API) are expected to use one for each record they access.

6.2.2 Obtaining Record Information

Naturally, software information *must* be scraped since it is unavailable through the zbMATH API; the remaining fields, however, *can* be accessed this way. Ultimately, it was decided that because a scraper was already running to obtain software information, it would be faster to collect the remaining fields the same way. Erroneous values caused by the scraping (particularly the *date* field as will be seen later) could be corrected during data cleaning using the zbMATH Open API.

6.2.3 Choice of Fields

Fields were chosen according to their relevance to a hypothetical data analysis task relating to software usage patterns. These are described in the following list, ordered by importance:

ID The unique DE number identifying the record being scraped

Software A list of all software cited in the record according to zbMATH

MSC The set of all mathematics subject classifications for the record

Date The year the paper was published

Language The language the article was written in

6.2.4 ID Choice

While exploring the structure of zbMATH articles, it became apparent that records could take multiple forms. This will be elaborated on during the implementation section as it caused various complications, though for the sake of the design its effect on ID choice was the most pertinent.

Records on zbMATH possess two separate ID's: a native Zbl code, as well as a universal DE number. DE number is harder to obtain using traditional browsing of the site, however it is the number both returned and required by zbMATH Open API requests. Additionally, all records on the platform possess DE numbers, but not all of them contain Zbl codes. To achieve standardisation, DE numbers were chosen over Zbl codes in all situations. The difference in ID structure can be demonstrated by the article in figure 2.

Zbl Code Zbl 1200.35057

DE Number 5797851

Importantly, the Scraper was designed to scrape the DE number rather than directly obtaining it from the page URL. This was chosen for consistency, since, although not used in our systems, it is possible to request the same page using its Zbl code instead of DE number.

6.2.5 MSC Code Detail Level

High detail MSC codes are available from articles, however filtering by set allows only the highest level of code to be used. Additionally, the finer granularity subjects are largely unnecessary. An example of this hierarchy can be seen in figure 4.

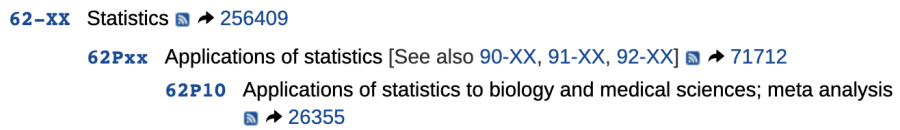


Figure 4: MSC hierarchy for applications of statistics

6.3 API

Essentially, this tool is a python accessible interface to the zbMATH website. It makes use of the underlying scraper tool to access the zbMATH records in a way that feels familiar to a web-based API. As we will see in this section however, it is not a traditional web-API. Instead, it takes advantage of the fact that users are in Python anyway to provide increased usability, simplicity, and performance.

The API forms the focus of primary requirement 2, and aimed to facilitate easy access to zbMATH data with minimal Python experience. This tool itself was designed as an easy interface to using both the Scraper and the zbMATH API to collect large amounts of data.

6.3.1 Aims

Intuitive Scraper Wrapper Easy access to the functionality of the scraper for various tasks.

Access to the zbMATH API The zbMATH API makes use of continuation tokens for large requests (such as when getting tens of thousands of ID's). These tokens can be difficult to use and expire within less than a second. As such, an easy wrapper for requests necessary for data analysis projects is imperative.

Out-of-the-box Usability Functions should be used with minimal tweaking. Functions for all basic AND aggregate features (ex. bundling requests) should be provided as well

Additionally, as per the requirements section, these tools should be in accord with the following four principles:

1. *Correctness:* The information obtained through the API should be correct.

2. *Repeatability:* The tools yield consistent datasets on the same machine.
3. *Replicability:* The tools yield consistent datasets on different machines.
4. *Reusability:* The tools are flexible in yielding requirement specific datasets.

6.3.2 Structure

Since the design of the API pivoted dramatically during implementation, the design and implementation sections are heavily intertwined. This section will attempt to outline and justify existing structural decisions, whereas the implementation section will explain the evolution of this design and the tools that inspired it.

6.3.3 Web API & Python API

Originally, the aim of the API was to mimic the design of the zbMATH Open API but tailored towards Python users, supplying only the functions necessitated by the end-users goals. Unfortunately, it became evident that this was largely insufficient, as users required the ability to write to files and interact with the data collection tools directly through Python. By routing queries through a locally hosted web API, the system was adding unnecessary layers of complication and completely negating the system's aim of easier access to the existing zbMATH systems. Due to this limited usability, the system was shifted to a Python package for easy interface with the required API calls (this was further split into two python clients, which will be addressed next). The ramifications of this change will be seen in the implementation section.

6.3.4 RESTful vs RESTless

At the same time the issues with the web API became apparent, so too did the problems with RESTfulness. For an API, being RESTful is generally prefered, as it ensures consistency in the results of function calls—an essential requirement for reproducible data analysis. While this is acceptable for basic functions which return small responses (such as getRecord or getIDCount), the same cannot be said for large-scale functions which collect massive amounts of data (getIdentifiers, scrapeRecords). For this, it became evident that storing responses in memory and returning a massive JSON object would not be in the nature of a REST API. One alternative would be to follow the footsteps of the zbMATH API and use continuation tokens. Unfortunately this would just be a pedantic attempt to cling to RESTfulness, when a superior alternative would be to abandon this restraint altogether. By taking advantage of the fact that we were no longer limited by being a web-API, we could provide built-in support for writing output and reading input from files. As such, the system was split into two components: an API client for all RESTful requests, effectively acting as a Python wrapper for common zbMATH API calls; and a Python client for all stateful changes such as writing and reading files. Since most data collection

tools require the storage of results anyway, providing these capabilities through built in functions achieves the aim of Out-of-the-box Usability.

6.3.5 Allowing Choice of Fields

One decision heavily related to the record scraper was whether to allow users to select which fields they wanted to store from the collected set of records. As noted earlier, there are four (non-id) fields we can scrape from any given record file: software, MSC, date, and language. For most purposes it was be unnecessary to use all these fields (e.g. the example analysis made no use of the language field). Originally, the plan was to allow the use of query parameters to select which of these fields to store. As with the Scraper design though, it was determined that since the records were already being accessed via scraping, the difference in speed caused by obtaining the remaining fields would be negligible. This decision was further supported by the fact that scraped records were of relatively insubstantial sizes, even at large scales.

6.3.6 Two Pronged Data Collection

Another large consideration of the system was how to collect record information. Succinctly, this is now done by collecting all necessary ID's directly through the zbMATH API, then subsequently iterating over the ID's and scraping them.

6.3.7 Quality-of-Life Functions

Since the API makes use of many long duration function calls, several convenience functions were added. These mainly take the form of continuation support. There are three supported features for this:

Scraper Continuation When record scraping, it is perfectly normal for the scraper to be terminated early, be it due to rate limits or the user themself. Due to the multi-hour nature of the scraping process though, it would be largely undesirable if the process had to be reset in these situations. As such, a function is supplied which checks the scraped records against the list of ID's to be scraped and continues scraping only on those not yet completed. This proved imperative in dealing with rate limits

ID Continuation While not directly supported, the system provides all the necessary features to update collections of ID's with more records. For example, if the user scraped records from 2015 to 2020, they may now wish to add papers from 2020-2022. Fortunately, by storing the output of the ID collection in JSON, several metadata fields (such as end date) are stored which can be checked later. Using this end date as the start for the next getIdentifiers call will effectively allow the user to supplement the previous list with all newer IDs. Additionally, since records are collected based on upload date (rather than publication date), there is no concern about new papers being uploaded to zbMATH within the previously scraped date range.

Limit Avoidance A breakthrough discovery allowed request limits to effectively be removed from the scraping process, cutting the data collection duration from days down to hours. This is natively supported through the Python client. Further documentation of this discovery will be discussed in the implementation section.

Data Cleaning The date field for scraped records has a high fault rate for erroneous collection due to the massive variation of zbMATH formatting. Despite attempts to avoid this, it was ultimately more convenient to access and update this information using the zbMATH Open API.

6.4 Data Collection & Analysis

As previously mentioned, the data analysis was an experiment demonstrating an example use case of the system. While the experimental design itself is worth considering, a comprehensive analysis of the experiment will be detailed in entirety at the end of the implementation section (for the reader's ease of readability). It is considered largely separate to the other phases of this project and should be examined as such.

7 Implementation

As mentioned in the design section, these tools were aimed towards Python users, so naturally all systems were developed through Python. Web scraping is performed through the BeautifulSoup framework due to its ease of use and network of supporting materials. This section details implementation specific considerations and other changes that arose during the process of implementing the aforementioned designs. As before, it will be split according to the individual components of the system.

7.1 Scraper

The Scraper is implemented as a Python class, with each instance created around a single record link. Each Scraper then uses BeautifulSoup to parse its pages HTML for a variety of fields on request (i.e. those listed in the design). All responses are output in JSON encodable formats, with the aggregate output using a Python dictionary for the datatypes natural integration with JSON.

7.1.1 Field Types & Examples

One important implementation decision was the storage type of several record fields. The table below indicates each attribute and its relevant data type.

Field	Data Type	Example
ID (DE Number)	Integer	5797851
Software	Dictionary {“ID”:“Name”}	{”554”: ”Mathematica”}
MSC	List [“MSC”]	[“35”, “33”, “34”, “42”, “65”]
Date	Integer	2010
Language	String	”English

Several of these choices may seem inconsistent at first, however each decision was carefully planned.

Integer ID Generally, storing identifying variables as an integer should be avoided, as it does not fit the semantics of the type. The primary concern is that an ID starting with zero cannot be stored or represented through an integer. Fortunately, zbMATH likely stores their IDs the same way, as prepending a zero to any ID results in the same page. Similarly, removing a zero from Zbl codes starting with zero yields the expected page. Hence IDs were stored as integers to save space.

String Software ID Unlike DE number, software ID is stored as a string. This is mainly because there is no standardisation to this ID, and it is simply the ID swMATH uses internally to link software. Little is known about its exact structure so the safer option of using a String was selected.

String MSC Unlike the above, MSC codes were stored as strings because of the additional set "JBM", which is evidently not a number.

Date Unlike the other options, storing year as an integer had no direct concerns. Instead the consideration was whether to collect the date from the zbMATH internal datestamp or by scraping the record directly. Obviously, the year the paper was published is more relevant for analysis purposes, however the ID collection filters by upload date (due to the structure of zbMATH's API). The publishing date was collected for these reasons.

Language The language of each article is scraped as raw text. It is not cleaned in any way which means many records have languages with "\n" and many space characters before their translation language. This field can be easily cleaned through the zbMATH API if needed.

7.1.2 Scraping ID

As mentioned during the design section, DE number is manually scraped from each article rather than taken from the URL provided to the Scraper. This was done for consistency since the URL may contain the Zbl code rather than DE number.

7.1.3 Accessing Date

Scraping the date of publication ended up being an extremely difficult feat, as there is no dedicated HTML tag for it. Additionally, it became clear during the large-scale testing of the system that no unified format for date exists. Currently the system splits records into two types, those with a cite button and those without. For those with cite buttons the date can generally be found in one of two locations, and this is checked using we scraping. If no cite button is present, many dates store the year of publication after the issue they were published in (see 2). This format can have either a period after it or no period.

These four cases cover over 99% of all zbMATH records, however that was still not enough. Even with REGEX matching, many erroneous dates slipped through large-scale testing. Since a majority of the program's runtime comes from making requests to zbMATH, further REGEX searches and fixes could be implemented with negligible impact to speed. Rather than spot-fix remaining differences though, it was at this point I decided to employ the zbMATH API. Cleaning the dataset using the internally stored Language & Date fields provides more accurate results than the scraper ever could. This resulted in the API's cleaning functionality.

7.1.4 Exception Handling

The API must be capable of running for hours on end, so being able to do so unsupervised was preferable. Beyond simply fixing errors and ensuring smooth runtime, exception handling was imperative. If a record fails to scraped properly,

the system handles this by making note of the issue then continuing with the remaining records. Assuming the number of exceptions is relatively low (this is true in practice), failed records can be manually handled later.

The next, much more significant issue is that of rate limits. Rather than having a traditional rate limit, zbMATH employs a flat threshold above which all subsequent requests are blocked. This was not discovered until extremely late into the project during the data collection phase and threatened the usability of the tools altogether. The specifics of this rate limit and dealing with it will be discussed later as it is worth its own section. For the purposes of exception handling though, whether this limit was exceeded was checked on the instantiation of a Scraper, and a custom LimitException was raised to allow the implementing code to handle it specially.

7.2 API

As seen in the design section, the API acts as the users interface to all their desired information from zbMATH. This functionality was generally split into two types: small functions which do not require interfacing with files, and large functions which do.

The smaller functions were bundled into the API client and are relatively simple in their implementations. Accessing individual records is done as a simple wrapper around the Scraper class, and other functions simply require parsing zbMATH API's XML responses using the lxml package's etree.

The Python client on the other hand is significantly more notable, with four large functions utilising many of the previously developed tools from the API client and the Scraper. These generally follow a single design pattern, formatting the output file with desired metadata then looping over desired records or ID pages to append them to the file.

The exact structure of the system can be seen in the `readme.md` in the appendices, however there are several notable elements which were implemented to ensure a smooth user experience. These will be addressed one by one later. First however, we will address the issues of the web API which preceded the current system.

7.2.1 Flask-RESTful API

As the first API component to be implemented, many of the fundamental components of the current systems were discovered here. The issues with the system were discovered when attempting to collect record IDs from the zbMATH API. As previously mentioned, zbMATH makes use of continuation tokens, which are appended as query tokens to subsequent requests for multi-page responses. Requests using these generally returned tens of thousands of IDs if not more, so using continuation features was understandably necessary. By noting the reasoning behind this decision though, it became clear that returning a massive JSON object of all ID's through the Flask API would be bad practice. Additionally, if the Flask API implemented continuation tokens, it would lose its

simplicity and no longer have any benefits over directly interfacing with the zbMATH API.

At this point, returning to the goals of the end user was the best way to determine the correct approach. Due to long runtimes, most users would opt to write the outputs of these requests to files anyway. This hinted that rather than a web API, a direct interface to zbMATH using Python would be a better approach.

Due to this pivot, the web API is largely incomplete and lacks many of the desired features of a web API, such as response headers and parameter validation. Fortunately, this implementation was not without purpose. By developing this web API, a deeper understanding of API structures and how they interact was obtained. Only by gaining this understanding did it become clear why web API's were not appropriate for this project. Additionally, this knowledge allowed exploration of zbMATH's API in much more depth. More than anything though, this change in direction had no drawbacks, as it occurred early in the project and all code was able to be directly transferred directly to pure Python functions.

7.2.2 API Client Considerations

Most considerations for the system at this phase related to the collection and storage of data, with the most important decisions all relating to data and files. Of important note is that unlike the Scraper, both of the client API's are not implemented as classes. This is because the functionality of the underlying methods require no state, so using classes would be an unnecessary attempt to cling to Object-Oriented Structure, wasting the flexibility of Python.

Output Format (JSON vs CSV)

The most pressing consideration of the API was the format of the output: JSON or CSV. Each had several strengths and weaknesses, so these points were laid out for comparison.

1. CSV
   (a) Lightweight
   (b) Native Python support for file streaming
   (c) Intuitive support with Pandas dataframes (data manipulation package)
   (d) No support for metadata
   (e) No nested lists
2. JSON
   (a) Native Python support through dictionaries
   (b) Support for nested lists

(c) Metadata storage possible
(d) Slightly larger file size
(e) No support for file streaming
(f) Manipulation required for use with Pandas

Output data had a naturally hierarchical structure, with lists of records containing further lists of software and MSC codes. As such, it was clearly necessary to use JSON storage over CSV, despite CSV's benefits for data analysis. Regardless, JSON does hold several strengths over CSV, most importantly being the native support for 'metadata' storage due to its hierarchical structure. Beyond this, file size differences were negligible for the data we worked with, and workarounds to the file streaming issues were both doable and unnecessary.

JSON File Builder

Due to the large quantities of data being collected, it would be difficult to store this in memory and perform all writes at the end. At least, this is what was expected when this module was created. During testing however, it became clear that file sizes were still relatively small, with around 10,000 records occupying only 1 megabyte. Despite this, performing writes at the end of the process would be dangerous if the program terminated early or raised any exceptions. As such, the JSON Builder was created.

The JSON Builder is a helper class which makes use of Pythons __enter__ and __exit__ functions to facilitate clean teardown of file streams when used with Pythons with statements. This guarantees files will be closed upon termination, and, in this circumstance particularly, that JSON files will be properly formatted upon closure. This class contains many functions for formatting JSON files with metadata like the filters of the ID/Record search, ID count, and so on. Additionally, the class allows IDs and records to be written to a properly formatted JSON file one at a time, allowing streamed output.

Several specific considerations are included to ensure safe file closure, as early termination is relatively common when dealing with rate limits and user termination. Beyond simply using Python's teardown functions, the class keeps a stack of necessary closing brackets ('}' and ']') and places them at the end if the function terminates early. With this, the only concern of early termination of the function is whether the class places commas before or after adding a new element (excluding the first and last elements as necessary), as trailing commas would lead to an improperly formatted JSON file. Hence, commas were deliberately placed after scraping a record but before writing it. This ensures that early termination would likely happen while record scraping or during some other long running function.

Parameter Type Handling

There are three parameters of concern for this API and those are the filters applied to ID collection:

Set The MSC code for the ID's to collect. (ex. '05')

Start The start date of the ID collection range. (ex. '1970-01-01T00:00:00Z')

End The end date of the ID collection range. (ex. '1970-01-01T00:00:00Z')

The system handles these the same way as the zbMATH API (quite literally). Since the API wraps the zbMATH API, it simply sends these parameters in the query directly and responds with zbMATH's response. For instance, if a string 'what' is passed as any of the parameters, the function simply raises a ValueError. On the other hand, if the end date is before the start date, the system raises no errors and simply returns 0 results. This was selected to ensure the API acted predictably to users of the zbMATH API.

Continuation

Continuation is a core functionality of the system. It provides the ability to resume a record scraping task from where it previously left off. Due to rate limits this functionality is literally essential to collecting large amounts of data. It operates by reading the IDs of all records which need to be scraped as well as those already scraped. Python sets are used to store these IDs as order is unnecessary. This makes removing scraped records extremely performant due to set operational complexity. From there, record scraping is performed normally with no noticeable impact on overall speed.

7.3 Rate Limits

When web scraping, most websites employ some form of rate limiting which restricts the number of requests that can be made in a certain amount of time. The necessity of these limits was addressed in the context survey. Unlike most websites though, zbMATH does not throttle requests using a rate limit, but instead employs a flat cutoff threshold above which the following is returned:

only 2000 requests per day per ip

Figure 5: zbMATH request threshold warning

As shown, this message declares that after 2000 requests the user must wait 24 hours before making subsequent requests. If this was truly the case, collecting even small ranges of records would require weeks. Fortunately, this message is not entirely truthful.

This first point of note is that the 2000 request limit is accurate. Attempts to limit the number of records scraped to nearly 2000 (using the limit argument in the suitable functions) run as expected. Beyond this however, we are met with the above image once again.

Our first breakthrough occurs by observing that the 24 hour rule is not accurate. While the exact window is still unknown, early attempts to run the

Scraper around 8 hours later were met with success. Subsequent testing over the span of a week while collecting data for the example analysis led to the discovery that the limit was likely around two hours. Occasionally shorter durations were noted but these were largely inconsistent. Additionally, making failing requests seems to extend the duration, as the reset time was noted to fail at two hours on occasion when the limit was raised within the previous hour or less.

The records from 2010 to 2015 for sets 11, 20, and part of 62 were collected manually using these 2000 record blocks, corresponding to. Each 2000 record block requires roughly 15 minutes allowing around five scrapes (10,000 records) per working day at most if reasonably spaced. This was already much more reasonable, however a small glitch was noted.

When performing the final scrapes of the first set, there were only 1,500 records or so remaining. To maximise efficiency, those records would be collected, then the scripts would be updated to collect for the new set. Unusually though, when the scripts for the next set were started, a full 2,000 records were collected. At first this was thought to have to do with not reaching the 2,000 record limit and the several minute delay between the scraper session. Many tests with shorter spaces between sessions were attempted, however these were risky due to the reset on cooldown caused by failed attempts. Upon completion of the next set though, this full reset of the cooldown occurred again. Unlike the previous time, I had begun logging the time of each scrape, and the distance between the sessions was noted to be eight minutes, far shorter than though previously possible.

This time, the connection between the shortened cooldown and the change in set was noted. It was theorised that the anomaly was caused by the ID collection phase that happened when scraping a new set. Upon trying to test this theory, the following was captured trying to make requests to the zbMATH API:

only 600 requests per minute per ip

Figure 6: zbMATH API request threshold warning

A completely different result from the previous attempt. Waiting several minutes though, the attempts to collect records worked and the API loaded as claimed. More unusually though, attempts to load the main site also worked. Through thorough testing of this, several findings were made. A summary of all findings thus far is listed below:

1. Triggering the 2000 request limit on the main site triggers the 600 request per minute on the zbMATH API as well.

2. Expiring the 600 request limit on the zbMATH API also expires the 2000 request limit on the main site.

3. The 600 request per minute ban lasts exactly 3 minutes.

4. The first request to zbMATH API after the ban expires is always denied, but the second works as expected.

5. The 2000 request per day ban lasts approximately 2 hours.

6. Making a request while IP banned restarts the ban timer.

Based on these findings, a theory was derived. Triggering either limit places an IP ban on the user who exceeded on both sites. The duration is dependent on the offence. IP ban information is globally stored, and exceeding the limit resets the ban, hence triggering a different ban (on the API site) overrides the 2 hour ban. When the 3 minute ban expires, it expires on both sites.

This works in practice and effectively allows unfettered data collection. This process was automated and is provided by the `fullScrape` function.

7.4 Testing

Since these tools work primarily by making requests to zbMATH, unit testing is not entirely feasible. Despite this, a collection of unit tests were written to test certain smaller features of the system. These could be potentially expanded in the future if necessary. Additionally, since the system works with live data, the best way to test its features was through large scale deployment. This was achieved during the data collection phase as a selection of over 50,000 records was scraped for the final result. Ultimately, since this is a data collecting tool (and a scraper particularly), there is necessarily going to be some noise in the data retrieved. Fortunately, if the software is scraped correctly it is possible to get accurate results for all remaining columns using the provided cleaning functions.

Due to this decision no testing summary is included, unit tests can be found in the test directory if required.

8 Experiment

As previously addressed, the experiment portion of this project was mainly used as a walkthrough for the systems tools. The results themselves are largely inconsequential to the primary objectives.

8.1 Collecting Data

Data collection serves as the most important part of this experiment, as it is the main function of these tools. Hence the selection steps are listed in detail. A Jupyter Notebook detailing the process is supplied along with the code (titled `experiment-log.ipynb`) which details all the programming dependent choices. Note that some choices rely on external resources such as swMATH and as such are not contained in the Jupyter Notebook.

8.1.1 Experiment Scoping

First, we define our question of interest. We are simply focused on exploring the data collected so a suitably open ended question will be sufficient for now.

How does the use of software vary between different mathematical subjects

With this immense level of vagueness we are free to explore the dataset openly while also having an idea of the ranges our essential attributes should take. More specifically, we know that we are interested in three attributes—software, publication date, and MSC codes—and that we should select a suitable range of these attributes in order to better variations and trends within them.

Aside - Limiting Runtime

Since record scraping is solely limited by the rate of HTTP requests to zbMATH, it is necessary to limit the number of records being collected. Regardless of the question being explored, there are two ways the user can achieve this:

Time Interval The range of dates across which to collect records from.

Class The mathematical subject (via MSC code) to collect records from.

Aside - Rate Limits

Note, this experiment was designed and executed before the rate limit skip discovery, as such it was essential to limit the number of records scraped to allow reasonable runtime. Even with the limited time interval selected, the process of scraping the first three sets took around 10 day while discovering the exact rate limit reset timer. Despite this limitation, data collection will be explained using the new tools.

8.1.2 Parameter Selection

The ability to distinguish between subjects is one of the primary benefits of this program. As such we opt to select several classifications of interest from before finding a time interval. This allows us to later select a time range in line with the number of posts in the selected subjects.

Subject Selection

In line with our initial question, our primary concern when selecting subjects is that they are sufficiently distinct so as to allow greater breadth of software choice. To select candidate subjects, we use a combination of domain knowledge as well as swMATH visualisations. Our process for selecting candidates follows:

1. Use `api_client`'s `getClasses` function to view a list of available set filters. (Alternatively, the user may visit zbMATH directly and browse the MSC codes.)
2. Browse swMATH in popular packages to find the most common MSC codes in varying disciplines.
3. Combine both lists to find a list of candidate MSC codes.

Using domain knowledge the following five subjects of interest were selected:

- 05 - Combinatorics: Requires software to enumerate permutations or graphs
- 11 - Number Theory: A field of pure maths dedicated to the study of integers
- 16 - Associative Rings & Algebras: A popular pure field of abstract algebra
- 20 - Group Theory & Generalisations: Another popular pure field of abstract algebra
- 62 - Statistics: A large, heavily practical branch of mathematics reliant on software

To check these fields utilise software and is sufficiently distinct, we then utilised swMATH's 'top MSC classes' list on certain software packages. As seen in the image below, statistics is the second most popular field to cite R in research papers. Using this on other packages—such as Numpy, CPLEX, etc.—we eliminated and added several subjects.

This resulted in the final list of five candidate subjects:

- 05 - Combinatorics: Requires software to enumerate permutations or graphs
- 11 - Number Theory: A field of pure maths dedicated to the study of integers

- 20 - Group Theory & Generalisations: Another popular pure field of abstract algebra
- 62 - Statistics: A large, heavily practical branch of mathematics reliant on software
- 65 - Numerical Analysis: An applied maths field centred in developing algorithms

Interval Selection

The next step of the process is to determine the acceptable range of dates for the system to scrape records from. First, the start date must be chosen according to the question being asked. For instance, if one was attempting to see the time evolution of software usage in a subject since its inception, selecting a start date before the software was popularised would be important. On the other hand, if one wished to look at the usage of software after it had become well established, selecting a more recent start is more important.

For our purposes, we simply wished to compare subjects' software usages over a reasonable range of years. As such, we selected a start date after most software packages were likely already well established: 2010. By arbitrarily testing end dates using the `api_client`'s `getIDCount` method, the final date of 2015 was selected. A five year period was the absolute minimum acceptable width for this question, however it became necessary due to the slow nature of the scraping process before the rate limit skip was discovered. This resulted in the five candidate sets having the following counts:

- 20: 15,243
- 11: 21,023
- 05: 34,450
- 62: 36,205
- 65: 49,759

Based on these final counts, the sets 20, 11, and 62 were selected due to their relatively lower counts and subject variety.

8.1.3 Data Collection

With the parameters selected, the data could now be collected. As mentioned earlier, the sets 20, 11, and 62 were scraped manually with the system restarted every two to four hours while awake. The total process took around 10 days with each set taking less time due to having greater understanding of the rate limits. For the purposes of this experiment though, the updated method for collection using the rate limit skip will be demonstrated on set 05.

ID & Record Scraping

Using the `fullCollect` method it is now extremely easy to collect data. The user simply has to use the function in any python interpreter they choose with the selected parameters. For this experiment, this was done in a simple script.

Cleaning

Data cleaning is provided by the cleanDataset function which allows the user to update the date and language fields directly from the zbMATH API. Two levels of rigour are provided in this function, with a strict clean updating all values as well as faster clean only updating records with obviously erroneous dates.

The strict clean makes requests to the zbMATH API for every record, updating dates for all documents and updating language if it is available. The fast clean checks every date to ensure it is within the 1900 - 2099, only checking with zbMATH if the value is outside this range. Additionally, the fast clean updates all language fields to remove the '\n' newline characters and anything afterwards. Experiments using the strict clean ran from 20:50 to 22:11, with a total runtime of over 81 minutes, whereas the fast clean finished execution in just 224 seconds at most.

8.2 Analysis

8.2.1 Visualisation Choice

By the principles of task abstraction set out by Munzner [6], we define our visualisation objectives. For this experiment, we are simply concerned with consuming the information for the sake of exploration, with an emphasis on comparison between MSC sets. For this purpose, an aligned bar chart would be a simple yet effective idiom. Specifically we choose two visualisations, the results of which are displayed in the next section:

A horizontally aligned stacked bar chart, where each bar represents one of the three chosen subjects. Glyphs in each bar represent a software package and vertical length channel encodes quantity.

As well as a horizontally aligned multi-view of three bar charts, where each view represents a horizontally aligned bar chart for a single subject. Bar charts show the number of citations for each top 3 package and others.

This is simple and does not convey much information about the data, but importantly the dataset transformations required for the visualisation display a common use case for the tools.

8.2.2 Data Transformation

The next step is to transform the data to be usable by matplotlib for these specific idioms. To do this we use a variety of dictionaries along with Python dictionary comprehension. The exact process is somewhat complicated and can

be found in the aforementioned Jupyter Notebook, however the process can be summarised as follows:

For each subject import dataset, count number of times each software is cited, find the top 3 software packages in each subject. Then find the following summary counts:

- Count for each top software package
- Count for all other software packages combined
- Count for all records with no software packages

Finally, format outputs for the exact visualisation required

8.2.3 Visualisation

Creating the final visualisations was relatively simple using matplotlb and the results can be seen in the aforementioned Jupyter Notebook.

8.2.4 Assessment

The visualisations are interesting to observe but provide relatively little information beyond overview statistics. Additionally, they make no use of time varying trends provided by the date fields. Since it does not account for date, many articles published before the advent of software in mathematics are present and may skew the proportions of software usage.

Ultimately, the visualisation itself was not the most important part of this sample experiment, but rather an avenue to demonstrate the data collection tools and their use in a practical context. In this regard it achieves its goals.

9 Evaluation & Critical Appraisal

At the beginning of this paper, three primary requirements and three secondary requirements were specified as outcomes of the project. The primary requirements were necessities that the project had to achieve in order to be considered successful. The secondary requirements were additional goals which, while nice to achieve, were not entirely necessary. As the project grew in scope, it became clear that achieving the primary goals would be more work than expected. Moreover, completing those requirements well would be even more important, as for any system prioritising user experience, the smaller details make most of the difference.

In this section, we will compare our results in comparison to the initial objectives and determine whether the project achieved these aims.

9.1 Primary Requirements

9.1.1 Requirement 1

Develop a web scraper capable of mass collection of software information from zbMATH records.

The web scraper produced is absolutely capable of collecting mass amounts of data as already displayed through its use collecting over 50,000 records in the sample analysis. Additionally, the scraper successfully collects all software used in these packages when present without fail.

9.1.2 Requirement 2

Develop an intuitive Python API for integrating the scraping tools into data analytics projects.

Likely the most important goal, this requirement was achieved to a higher level than any of the other tasks. The systems themselves are already immensely intuitive, with complex aggregate functions presented in a simplified manner to facilitate the exact requirements of the typical user. Small additions such as data cleaning and continuation tools allow the user to perfect their datasets without needing any additional tweakage. Users who still want to manually change components however still have this option, as the system can be decomposed into a collection of completely independent functions of varying levels of abstraction.

Beyond just the API itself though, the structure of the project itself—with setup, requirements, and readme files—allows the tools to become an easily installable Python Package. This combination of small details ultimately leads to a smoother user experience from setup to implementation.

This API should facilitate procedures following four principles of reproducibility

Correctness

By using the data cleaning tools the information accessed by the system is absolutely correct, eliminating any issues caused by scraping. Additionally, software was stored in a specially separated HTML tag so no issues could have arisen in its collection. One concern of note though is that records are filtered based on upload date, but information collected is concerning publication date. This is relatively inconsequential if collecting all records from a given start date to the present, but if only collecting ID's on a small range of dates it is possible to exclude some records that were uploaded at a later date. This is an avoidable issue and with more time systems could likely be created to counter it, however it is still an important flaw of the system to address.

Repeatability

By nature of the stateless implementation of the system, the tools will always yield the same results when performed on the same machine. Contrary to the previous principle, this is where the upload date filter is beneficial to the system, as even retroactive additions to the zbMATH database will not alter the results of the system given a date interval.

Replicability

Again, using the tools is stateless and simply connects to the zbMATH database. Additionally, the setup.py and requirements.txt files ensure any system can respectively deploy or develop the tools with identical results.

Repeatability

Thanks to the ability to specify set and date filters on the collected input ID's, the tools are capable of providing a variety of datasets easily and efficiently.

9.1.3 Requirement 3

Demonstrate the use of the aforementioned tools through a small scale procedure.

The small scale procedure was explained in detail following the implementation section. This outline demonstrated the exact use case for the tools from experiment inception to analysis, demonstrating the efficacy of the tools in practical applications.

9.2 Secondary Requirements

9.2.1 Requirement 4

Adapt the API to allow automated, unchecked collection of data beyond rate limits.

Initially thought to be an unattainable reach goal, this is now the most impressive feature of the system. By overriding internal IP bans the system is able to trick zbMATH and bypass limits entirely, significantly upgrading the usability of the API in large scale collection projects.

9.2.2 Requirement 5

The example procedure should yield a comprehensive dataset capable of reuse in other data analysis experiments.

While certainly achieved to a certain extent, the datasets produced for the experiment can by no means be considered comprehensive. Spanning only five years and four subjects, the datasets are likely useful for exploratory purposes only. Largely due to the timing of the rate limit discovery, this is an acceptable loss. Fortunately, this can not be considered a complete failure as the issues that caused this failure were ultimately rectified, and systems demonstrated show that a fully comprehensive data set for a single subject spanning 20 years may be collectable in less than a day.

9.2.3 Requirement 6

While certainly not to as much of an extent as the first four requirements, this goal was inarguably completed. Visualisation techniques and principles specified by Munzner [6] were employed so as to ensure a both effective and expressive visualisation of the created datasets. While the visualisations were strong though, they were certainly not creative, using common popular idioms and basic spatial layouts. This is expected of Jupyter Notebooks though as its usage is better suited towards analysis and trend spotting than the production of powerful and thought out visualisation. Ultimately this goal was a success.

sections/conclusion

References

[1] zbmath about.

[2] zbmath open api.

[3] ice-mc2 zbmath api. `https://github.com/ice-mc2/zbmath-api`, 2019.

[4] zbmath open links api. `https://github.com/zbMATHOpen/linksApi`, 2022.

[5] Daniel Glez-Peña, Anália Lourenço, Hugo López-Fernández, Miguel Reboiro-Jato, and Florentino Fdez-Riverola. Web scraping technologies in an api world. *Briefings in bioinformatics*, 15(5):788–797, 2014.

[6] Tamara Munzner. *Visualization analysis and design*. CRC press, 2014.

[7] Carlos Rodríguez, Marcos Baez, Florian Daniel, Fabio Casati, Juan Carlos Trabucco, Luigi Canali, and Gianraffaele Percannella. Rest apis: a large-scale analysis of compliance with principles and best practices. In *International conference on web engineering*, pages 21–39. Springer, 2016.

10 Appendix A - Ethics

UNIVERSITY OF ST ANDREWS
TEACHING AND RESEARCH ETHICS COMMITTEE (UTREC)
SCHOOL OF COMPUTER SCIENCE
PRELIMINARY ETHICS SELF-ASSESSMENT FORM

This Preliminary Ethics Self-Assessment Form is to be conducted by the researcher, and completed in conjunction with the Guidelines for Ethical Research Practice. All staff and students of the School of Computer Science must complete it prior to commencing research.

This Form will act as a formal record of your ethical considerations.
Tick one box

Staff Project
Postgraduate Project
✓ **Undergraduate Project**

Title of project

Reproducible Data Analytics

Name of researcher(s)

Jefferson Orion Portelli

Name of supervisor (for student research)

Alexander Konovalov

OVERALL ASSESSMENT (to be signed after questions, overleaf, have been completed)

Self audit has been conducted **YES** **NO**

There are no ethical issues raised by this project

Signature Student or Researcher

Print Name

ORION PORTELLI

Date

22/09/2021

Signature Lead Researcher or Supervisor

Print Name

Alexander Konovalov

Date

23/09/2021

This form must be date stamped and held in the files of the Lead Researcher or Supervisor. If fieldwork is required, a copy must also be lodged with appropriate Risk Assessment forms. The School Ethics Committee will be responsible for monitoring assessments.

Computer Science Preliminary Ethics Self-Assessment Form

Research with human subjects

Does your research involve human subjects or have potential adverse consequences for human welfare and wellbeing?

YES NO

If YES, full ethics review required
For example:
Will you be surveying, observing or interviewing human subjects?
Will you be analysing secondary data that could significantly affect human subjects?
Does your research have the potential to have a significant negative effect on people in the study area?

Potential physical or psychological harm, discomfort or stress

Are there any foreseeable risks to the researcher, or to any participants in this research?

YES NO

If YES, full ethics review required
For example:
Is there any potential that there could be physical harm for anyone involved in the research?
Is there any potential for psychological harm, discomfort or stress for anyone involved in the research?

Conflicts of interest

Do any conflicts of interest arise?

YES NO

If YES, full ethics review required
For example:
Might research objectivity be compromised by sponsorship?
Might any issues of intellectual property or roles in research be raised?

Funding

Is your research funded externally?

YES NO

If YES, does the funder appear on the 'currently automatically approved' list on the UTREC website?

YES NO

If NO, you will need to submit a Funding Approval Application as per instructions on the UTREC website.

Research with animals

Does your research involve the use of living animals?

YES NO

If YES, your proposal must be referred to the University's Animal Welfare and Ethics Committee (AWEC)

University Teaching and Research Ethics Committee (UTREC) pages

11 Appendix B - Instructions

See readme